

Monepha Mitchell
ITAI 1371 – Machine Learning
Final Machine Learning Project
August 7, 2026

Executive Summary

The purpose of this project was to develop and compare machine-learning models that predict whether a telecommunications customer will leave the company. Customer churn is a classification problem because the target variable contains two categories: customers who churned and customers who did not churn.

The Telco Customer Churn dataset contained 7,043 customer records and 21 original columns. I cleaned the dataset, corrected the TotalCharges column, removed the unique customer identification column, and created two new features called TenureGroup and NumberOfServices. I divided the data into 70% training, 15% validation, and 15% testing sets using stratification to preserve the original class distribution.

Six required classification models were trained: Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, K-Nearest Neighbors, and Support Vector Classifier. I also created an Average Ensemble and a Bayesian Ensemble using the three strongest individual validation models.

Gradient Boosting was the strongest individual validation model, with a validation ROC-AUC of 0.8376. The Bayesian Ensemble produced the strongest final test result, with an accuracy of 0.7711, precision of 0.5470, recall of 0.7893, F1-score of 0.6462, and ROC-AUC of 0.8531. These results show that combining multiple models can improve the ability to identify customers who may churn.

Introduction

Customer churn occurs when a customer stops using a company's products or services. Churn can affect a telecommunications company's revenue because the company may lose monthly payments and must spend additional resources to replace departing customers. A machine-learning model can help a company identify customers who have a higher probability of leaving so that retention efforts can be directed toward them.

The objective of this project was to develop a binary-classification system that predicts customer churn. The project compared six individual models and two ensemble approaches. The models were evaluated using accuracy, precision, recall, F1-score, and ROC-AUC.

The main research question was:

Which individual or ensemble machine-learning model provides the strongest performance for predicting customer churn?

A second goal was to determine whether combining the predictions of the strongest individual models would improve performance.

Dataset Description

The project used the Telco Customer Churn dataset. The dataset contained 7,043 customer records and 21 original columns. The target variable was Churn, which contained the values Yes and No.

The dataset included several types of customer information:

- Demographic information, such as gender, senior-citizen status, partner status, and dependents
- Account information, such as tenure, contract type, payment method, and paperless billing
- Service information, such as phone service, internet service, online security, technical support, and streaming services
- Billing information, such as monthly charges and total charges
- The target variable showing whether the customer churned

The original target distribution was imbalanced. 5,174 customers did not churn and 1,869 customers who churned. This represented approximately 73.46% non-churn customers and 26.54% churn customers.

Data Cleaning

I first inspected the dataset for missing values, duplicate rows, and incorrect data types. Pandas originally reported no standard missing values. However, further inspection showed that the TotalCharges column contained 11 blank entries and was stored as an object instead of a numerical column.

I converted TotalCharges into a numerical data type using `pd.to_numeric()` with invalid entries changed to missing values. All 11 affected customers had a tenure value of zero, which indicated that they were new customers who had not yet accumulated total charges. I therefore replaced the missing TotalCharges values with zero.

After this correction, TotalCharges was stored as a floating-point number, and no missing values remained.

I also removed the customerID column from the modeling dataset. This column uniquely identified each customer but did not describe customer behavior. Using it as a predictor would not provide a meaningful pattern that could be applied to new customers.

After cleaning, the dataset contained 7,043 rows and 20 columns.

Feature Engineering

I created two additional features to provide the models with more useful information.

TenureGroup

The original tenure column contained the number of months that each customer had remained with the company. I divided the values into four groups:

- New Customer: 0–12 months
- Developing Customer: 13–24 months
- Established Customer: 25–48 months
- Long-Term Customer: 49–72 months

The TenureGroup feature may help the models recognize differences between newer and longer-term customers. New customers may have weaker relationships with the company, while customers who have remained for several years may be more connected to its services.

The tenure groups contained:

- 2,186 new customers
- 1,024 developing customers
- 1,594 established customers

- 2,239 long-term customers

NumberOfServices

I also created the NumberOfServices feature. This feature counted the services marked Yes for each customer across eight service columns:

- Phone service
- Multiple lines
- Online security
- Online backup
- Device protection
- Technical support
- Streaming television
- Streaming movies

The number of services ranged from zero to eight. Customers used an average of approximately 3.36 services. This feature was created because customers who use several services may be more connected to the company than customers who use only one service.

After feature engineering, the dataset contained 22 columns.

Train, Validation, and Test Split

The target variable was numerically encoded as:

- 0 = No churn
- 1 = Churn

The predictor variables were stored in X, while the encoded target was stored in y.

I divided the dataset into:

- 4,929 training records, or approximately 70%
- 1,057 validation records, or approximately 15%
- 1,057 test records, or approximately 15%

I used stratified splitting to preserve the original churn percentage in each dataset. The full dataset had a churn rate of approximately 26.54%. The training, validation, and testing sets each had a churn rate close to this percentage.

The training set was used to teach the models. The validation set was used to compare models and select the strongest approaches. The test set remained untouched until the final evaluation. This separation helped prevent the test results from influencing model selection.

Data Preprocessing

The predictor dataset contained five numerical columns:

- SeniorCitizen
- tenure
- MonthlyCharges
- TotalCharges
- NumberOfServices

It also contained 16 categorical columns.

I used StandardScaler to standardize the numerical columns. Standardization placed numerical values on a similar scale, which is especially important for models such as Logistic Regression, K-Nearest Neighbors, and Support Vector Classifier.

I used OneHotEncoder to convert the categorical values into numerical indicator columns. The encoder used `handle_unknown="ignore"` so that an unfamiliar category would not cause an error during prediction.

The preprocessing process converted the original 21 predictor columns into 50 processed features. The preprocessing tools were fitted only on the training data. The same fitted transformations were then applied to the validation and test sets. This prevented information from the validation or test sets from leaking into the training process.

Class Balancing

Before balancing, the training set contained:

- 3,621 non-churn customers
- 1,308 churn customers

Because the minority class was much smaller, a model could achieve a relatively high accuracy by predicting mostly non-churn customers. However, such a model would fail to identify many customers who were actually at risk of leaving.

I used RandomOverSampler to balance only the training data. The sampler duplicated minority-class training observations until both classes contained 3,621 records. The balanced training dataset therefore contained 7,242 observations.

The validation and test sets were not balanced. They retained the original class distribution so that their results would better represent the real dataset.

Machine-Learning Models

Six required classification models were trained.

Logistic Regression

Logistic Regression served as a strong and interpretable baseline model. It estimates the probability that a customer belongs to the churn class based on a weighted combination of the predictor variables.

The model used a maximum of 2,000 iterations to allow the training process to converge.

Decision Tree Classifier

The Decision Tree divided the data into branches based on feature values. It can capture nonlinear relationships and is relatively easy to interpret.

The tree used a maximum depth of six and required at least ten observations in each leaf. These limits were included to reduce the possibility of creating an overly complex tree.

Random Forest Classifier

Random Forest combined predictions from 250 decision trees. Each tree was trained using variations of the data and features. Combining many trees can produce more stable predictions than relying on one decision tree.

The model used a maximum depth of ten and a minimum of three observations per leaf.

Gradient Boosting Classifier

Gradient Boosting created a sequence of decision trees. Each new tree attempted to correct errors made by the previous trees.

The model used 150 estimators, a learning rate of 0.05, and a maximum tree depth of three.

K-Nearest Neighbors Classifier

K-Nearest Neighbors classified each customer based on similar customers in the training data. The model used 15 neighbors and distance-based weighting, meaning closer neighbors had a stronger influence on the prediction.

Support Vector Classifier

The Support Vector Classifier used a radial-basis-function kernel to identify a boundary between churn and non-churn customers. Probability estimation was enabled so that ROC-AUC and ensemble probabilities could be calculated.

Evaluation Metrics

Each model was evaluated using five metrics.

Accuracy measured the percentage of all predictions that were correct.

Precision measured how often customers predicted to churn actually churned. Higher precision reduces unnecessary retention efforts directed toward customers who were not likely to leave.

Recall measured the percentage of actual churn customers identified by the model. Recall was especially important because a false negative represented a customer who was likely to leave but was not identified.

F1-score balanced precision and recall into one measurement.

ROC-AUC measured the model's ability to rank customers by churn risk across different classification thresholds. A higher ROC-AUC indicated stronger separation between churn and non-churn customers.

Validation Results

The models were first compared on the validation dataset.

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Gradient Boosting Classifier	0.7616	0.5351	0.7865	0.6369	0.8376
Bayesian Ensemble	0.7701	0.5503	0.7402	0.6313	0.8374
Average Ensemble	0.7569	0.5302	0.7509	0.6215	0.8373
Random Forest Classifier	0.7616	0.5381	0.7295	0.6193	0.8329
Logistic Regression	0.7531	0.5237	0.7865	0.6287	0.8319
Decision Tree Classifier	0.7408	0.5085	0.7473	0.6052	0.8144
Support Vector Classifier	0.7417	0.5097	0.7473	0.6061	0.8069
K-Nearest Neighbors Classifier	0.6859	0.4452	0.7367	0.5550	0.7769

Gradient Boosting was the strongest individual validation model. It produced the highest validation ROC-AUC of 0.8376 and the highest individual F1-score of 0.6369.

Logistic Regression had the same recall as Gradient Boosting, identifying approximately 78.65% of validation customers who churned. However, Gradient Boosting had stronger accuracy, precision, F1-score, and ROC-AUC.

K-Nearest Neighbors produced the lowest validation performance. Its ROC-AUC was 0.7769, and its accuracy was 0.6859. Distance-based models can have difficulty when the dataset contains many one-hot-encoded features because the meaning of distance may become less clear as the number of dimensions increases.

Ensemble Models

The strongest three individual validation models were:

1. Gradient Boosting Classifier
2. Random Forest Classifier
3. Logistic Regression

I used these models to create two ensemble approaches.

Average Ensemble

The Average Ensemble calculated the churn probability from each of the three selected models and then averaged the three probabilities. Customers with an average probability of at least 0.50 were classified as churn customers.

The Average Ensemble achieved a validation ROC-AUC of 0.8373. This was close to the Gradient Boosting result but did not exceed it.

Bayesian Ensemble

The Bayesian Ensemble used Gaussian Naive Bayes as a meta-model. Five-fold stratified cross-validation was used to create out-of-fold training probabilities from the three selected models. These probabilities became the input features for the Bayesian meta-model.

Unlike equal averaging, the Bayesian approach learned a relationship between the base-model predictions and the final churn outcome. It achieved a validation accuracy of 0.7701, which was the highest validation accuracy among the evaluated approaches.

Final Test Results

After model selection was completed, the models were evaluated on the untouched test dataset.

The Bayesian Ensemble produced the strongest final ROC-AUC:

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Bayesian Ensemble	0.7711	0.5470	0.7893	0.6462	0.8531
Gradient Boosting Classifier	0.7512	0.5191	0.8250	0.6372	0.8517
Random Forest Classifier	0.7644	0.5385	0.7750	0.6354	0.8482
Logistic Regression	0.7294	0.4934	0.8036	0.6114	0.8482
Decision Tree Classifier	0.7502	0.5192	0.7714	0.6207	0.8307
Support Vector Classifier	0.7446	0.5116	0.7857	0.6197	0.8214
K-Nearest Neighbors Classifier	0.6944	0.4544	0.7643	0.5699	0.7860

The separate model-comparison PDF contains the corrected complete table, including the Average Ensemble test result.

The Bayesian Ensemble achieved:

- Accuracy: 0.7711
- Precision: 0.5470
- Recall: 0.7893
- F1-score: 0.6462
- ROC-AUC: 0.8531

The model correctly identified 221 of the 280 test customers who churned. It missed 59 churn customers. It correctly classified 594 of the 777 non-churn customers and incorrectly predicted churn for 183 non-churn customers.

The confusion matrix was:

	Predicted No Churn	Predicted Churn
Actual No Churn	594	183
Actual Churn	59	221

The model's recall of 0.7893 means that it identified approximately 79% of customers who actually churned. Its precision of 0.5470 means that approximately 55% of its churn predictions were correct.

Best Model Analysis

The Bayesian Ensemble was selected as the best final model because it produced the highest test ROC-AUC and F1-score. It performed better than the individual models by learning how to combine the probabilities produced by Gradient Boosting, Random Forest, and Logistic Regression.

These three models have different strengths. Logistic Regression models broad linear relationships, Random Forest averages predictions from many trees, and Gradient Boosting creates trees that correct earlier errors. Combining these different approaches allowed the Bayesian Ensemble to use information from several modeling strategies.

Although Gradient Boosting had a slightly higher validation ROC-AUC, the Bayesian Ensemble performed best on the final test set. Its test ROC-AUC increased to 0.8531, compared with 0.8517 for Gradient Boosting.

Gradient Boosting had the highest test recall of 0.8250, meaning it identified more churn customers. However, the Bayesian Ensemble provided a better overall balance between accuracy, precision, recall, F1-score, and ROC-AUC.

Weakest Model Analysis

K-Nearest Neighbors was the weakest model on both validation and testing. It produced a validation ROC-AUC of 0.7769 and a test ROC-AUC of 0.7860.

One possible reason is that K-Nearest Neighbors depends heavily on distances between observations. After one-hot encoding, the dataset contained 50 processed features. In a higher-dimensional feature space, it can become more difficult for a distance-based model to determine which customers are truly similar.

K-Nearest Neighbors also had the lowest precision, meaning a larger portion of its churn predictions were incorrect.

Limitations

This project had several limitations.

First, the models used selected hyperparameters rather than a large automated hyperparameter search. Other combinations of parameters may produce stronger performance.

Second, RandomOverSampler balanced the training data by duplicating minority-class observations. Although this improved class balance, duplicated records do not create new customer patterns.

Third, all predictions used a classification threshold of 0.50. A telecommunications company may prefer a different threshold, depending on the cost of losing a churned customer compared with the cost of contacting a customer who would not churn.

Fourth, the project used one dataset. The results may not automatically apply to a different telecommunications company or a different time period.

Finally, the models identified relationships in the data but did not prove that any feature directly caused customer churn.

Future Improvements

Future work could include:

- Using GridSearchCV or RandomizedSearchCV to tune model hyperparameters
- Comparing RandomOverSampler with SMOTE or class-weighted models
- Calibrating the predicted probabilities
- Testing different classification thresholds
- Using SHAP values to explain individual predictions
- Applying cost-sensitive evaluation based on retention expenses
- Testing the models on data from another telecommunications company
- Retraining the model as customer behavior changes

These improvements could make the model more accurate, more explainable, and more useful for business decisions.

Conclusion

This project developed a complete machine-learning workflow for predicting telecommunications customer churn. The process included data cleaning, feature engineering, stratified data splitting, numerical scaling, categorical encoding, training-only class balancing, individual model comparison, ensemble development, and final test evaluation.

Gradient Boosting was the strongest individual validation model, with a ROC-AUC of 0.8376. K-Nearest Neighbors produced the weakest results.

The Bayesian Ensemble was the strongest final model. It achieved an accuracy of 0.7711, precision of 0.5470, recall of 0.7893, F1-score of 0.6462, and ROC-AUC of 0.8531 on the untouched test set.

The results show that customer information related to account history, services, contracts, and billing can be used to estimate churn risk. The final model could potentially help a

telecommunications company prioritize customers for retention efforts. However, the predictions should support human decision-making rather than replace it.

Overall, this project demonstrated the importance of careful preprocessing, separate validation and test sets, multiple evaluation metrics, and the comparison of both individual and ensemble machine learning models.

References

Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18(17), 1–5. <https://jmlr.org/papers/v18/16-365.html>

pandas development team. (2026). *pandas documentation*. <https://pandas.pydata.org/docs/>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., VanderPlas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(85), 2825–2830. <https://www.jmlr.org/papers/v12/pedregosa11a.html>

Python Software Foundation. (2026). *Python programming language*. <https://www.python.org/>

Scikit-learn developers. (2026). *Scikit-learn: Machine learning in Python*. <https://scikit-learn.org/stable/>